



EXPERIMENT NO: 9

Student Name: Aditya Singh
Branch: BE CSE
Semester: 6th
Subject Name: System Design

UID: 23BCS11827
Section/Group: KRG 2B
Date of Performance: 16/04/2026
Subject Code: 23CSH-314

Aim: To design a scalable platform that allows the users to book and discover tickets for various movies, concerts and live events.

Functional Requirements

1. Users should be able to search an event based on some title, location and date.
2. Users should be able to view the details and description of the events like seat availability, details, metadata (like duration, cast and ratings etc)
3. Users should be able to book the ticket for the show if tickets/seat are available.

Non Functional Requirements

1. The system should support 100 Million DAU (Daily Active Users).
2. CAP Theorem:
The system should be highly available when users are searching or viewing an event.
The system should be highly consistent with respect to booking a specific ticket.
3. The system should have a low latency and should respond in milliseconds.

Core Entities

User: Identity and profile information.
Event: The actual show/movie/concert.
Show: A specific instance of an event.
Seat: Individual units within a venue linked to a Show.
Booking: The transaction linking a User to a Seat.

API Design

1. Users & Auth:

POST: /api/v1/auth/signup
POST: /api/v1/auth/login
GET: /api/v1/users/profile [User profile & booking history]

2. Discovery (Events & Shows):

GET: /api/v1/events [Search with filters: title, city, date]
GET: /api/v1/events/{event_id} [Comprehensive event details & metadata]



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

GET: /api/v1/events/{event_id}/shows [List of shows for a specific event]

GET: /api/v1/shows/{show_id}/seats [Real-time seat availability map]

3. Booking & Transactions:

POST: /api/v1/bookings/reserve [Temporary lock on seats for 5-10 mins]

POST: /api/v1/bookings/confirm [Finalize booking after payment]

GET: /api/v1/bookings/{booking_id} [Check booking status/get e-ticket]

DELETE: /api/v1/bookings/{booking_id} [Cancel booking/release seats]

4. Venues:

GET: /api/v1/venues/{venue_id} [Venue details and location]

GET: /api/v1/venues/{venue_id}/layout [Static seat map structure]

Database Schema Design

1. Users Table

user_id (UUID, PK)

name, email, password_hash

created_at (Timestamp)

2. Events Table

event_id (UUID, PK)

title (Varchar, Indexed)

description (Text)

metadata (JSONB) [Stores: cast, duration, language, ratings]

3. Shows Table

show_id (UUID, PK)

event_id (FK)

venue_id (FK)

start_time (Timestamp, Indexed)

city (Varchar, Shard Key)

4. Show_Seats Table (The Transactional Table)

show_seat_id (UUID, PK)

show_id (FK)

seat_number (Varchar)

status (Enum: Available, Locked, Booked)

price (Decimal)

version (Int) [For Optimistic Locking]

5. Bookings Table

booking_id (UUID, PK)

user_id (FK)

show_id (FK)

status (Enum: Pending, Confirmed, Cancelled)

booking_time (Timestamp)

High Level Design

